

Documenttype: Verbetervoorstellen & Toekomstige Implementaties

Projectnaam: TaskManagerMaeconomy

1. Inleiding

Dit document bevat een diepgaande analyse van de testresultaten, concrete verbetervoorstellen en de bijbehorende broncode-optimalisaties voor het project **TaskManagerMaeconomy**. Tijdens de testfase is de applicatie kritisch beoordeeld op zowel functionele als non-functionele eisen. Hoewel de kernarchitectuur (waaronder de Firebase-authenticatie en de data-isolatie) stabiel is gebleken, heeft het testprotocol enkele kwetsbaarheden in de invoervalidatie en runtime-state aan het licht gebracht. Dit rapport documenteert de oplossingen voor deze gefaalde testcases en formuleert een roadmap voor toekomstige software-uitbreidingen.

2. Invoervalidatie en Foutafhandeling (Oplossing voor TC-B02)

2.1 Analyse van de huidige code

In de initiële ontwikkelfase was binnen de database-service de volgende logica geprogrammeerd voor het aanmaken van taken:

```
TypeScript
if (!title || title.trim() === "") {
  title = "Untitled Task";
}
```

Architectonisch probleem: Deze opzet corrigeert een foutieve of lege invoer aan de backend-zijde in plaats van deze rigoures te weigeren. Dit verklaart de status `Fail` bij testcase TC-B02: de applicatie onderbreekt de actie niet, maar omzeilt het validatieprobleem door een fallback-waarde toe te kennen en een leeg item naar de database te pushen. Dit leidt tot datavervuiling binnen Firebase Firestore en een verminderde User Experience (UX).

2.2 Verbetervoorstel: Exception Handling

Om te voldoen aan professionele programmeerstandaarden moet de logica worden aangepast naar *Strict Validation*. In plaats van de variabele `title` te muteren, dient de functie een expliciete foutmelding (`Error`) op te gooien. Hierdoor wordt de asynchrone keten direct afgebroken en krijgt de frontend de kans om hier adequaat op te reageren.

2.3 Gerealiseerde Code-optimalisatie (Database Service)

TypeScript

```
export const createTask = async (
  uid: string | undefined,
  title: string,
  date: Date | undefined,
  categoryId: string
) => {
  // Rendementscontrole op gebruikerssessie
  if (!uid) return null;

  // Strict Input Validation: Voorkom lege strings en spaties
  if (!title || title.trim() === "") {
    throw new Error("Task title cannot be empty");
  }

  // Hier volgt de stabiele Firebase Firestore push-logica...
};
```

2.4 Frontend Implementatie (UI-laag)

In het scherm of de component waar de gebruiker de taak toevoegt, wordt de asynchrone aanroep nu ingepakt in een try/catch-blok. Dit stelt de applicatie in staat om runtime-errors op te vangen en visueel terug te koppelen:

JavaScript

```
const handleAddTask = async () => {
  try {
    await createTask(uid, taskTitle, date, categoryId);
    setTaskTitle(""); // Reset invoerveld bij succes
  } catch (error) {
    // Gebruikersvriendelijke feedback in de UI
    alert("Vul een taakomschrijving in voordat u deze probeert toe te voegen.");
  }
};
```

3. Evaluatie en Conclusie van de Testcyclus

Uit de algehele testmatrix blijkt dat de fundamentele bouwstenen van de applicatie solide zijn opgezet. De kritieke testgevallen omtrent security en privacy – met name de authenticatie-flows (TC-A01 t/m TC-A03) en de strikte scheiding van gebruikersdata (User-

scoping via TC-C02) – hebben allemaal de status Pass behaald. Dit bewijst dat de applicatie voldoet aan de basiseisen rondom databeveiliging en de AVG-richtlijnen. De geconstateerde imperfecties bevinden zich louter op het niveau van invoersanitatie en reactieve state-updates in de frontend (zoals gebleken bij TC-B03). De toegepaste tests hebben hiermee exact hun doel bereikt: het tijdig blootleggen van logische randgevallen voordat de software in een productieomgeving wordt opgeleverd.

4. Toekomstige Uitbreidingen en Structurele Verbeteringen

Om de softwarekwaliteit, schaalbaarheid en functionaliteit van TaskManagerMaeconomy naar een hoger niveau te tillen, zijn de volgende drie structurele optimalisaties geformuleerd voor de volgende ontwikkelcyclus:

4.1 Implementatie van Firestore Offline Persistence

- **Doel:** Verhogen van de applicatie-beschikbaarheid en robuustheid.
- **Toelichting:** Momenteel is de app volledig afhankelijk van een actieve netwerkverbinding. Wanneer de verbinding wegvalt, kan de app haperen of tijdelijk geen data tonen. Door de ingebouwde *Offline Persistence* van de Firebase SDK te activeren, cachet de applicatie de Firestore-data lokaal op het toestel. Gebruikers kunnen offline taken blijven toevoegen of wijzigen; zodra het toestel weer verbinding maakt met het internet, synchroniseert Firebase alle wijzigingen geruisloos op de achtergrond.

4.2 Centralisatie van de Applicatiestate (Context API / Redux)

- **Doel:** Verbeteren van de code-onderhoudbaarheid en het elimineren van *prop-drilling*.
- **Toelichting:** Nu de app functioneel complexer wordt, neemt het risico toe dat de state (zoals gebruikersgegevens en actieve taaklijsten) te gefragmenteerd raakt over losse componenten. Door een centrale state management-laag te introduceren middels de React Context API of Redux Toolkit, ontstaat er één *Single Source of Truth*. Dit sluit direct aan bij de Object-Oriented Programming (OOP) principes door data en logica logisch te ontkoppelen.

4.3 Introductie van een Geautomatiseerde Validatie-laag (Formik & Yup)

- **Doel:** Uniforme en waterdichte invoervalidatie over de gehele applicatie.
- **Toelichting:** Om handmatige checks zoals `.trim() === ""` in de toekomst te automatiseren, verdient het aanbeveling om formulierbeheer te professionaliseren met libraries zoals Formik in combinatie met Yup schema-validatie. Hiermee kunnen complexe restricties (zoals een maximaal aantal tekens voor een taak of

strikte e-mailpatronen bij registratie) declaratief worden vastgelegd, waardoor de kans op runtime-bugs drastisch afneemt.